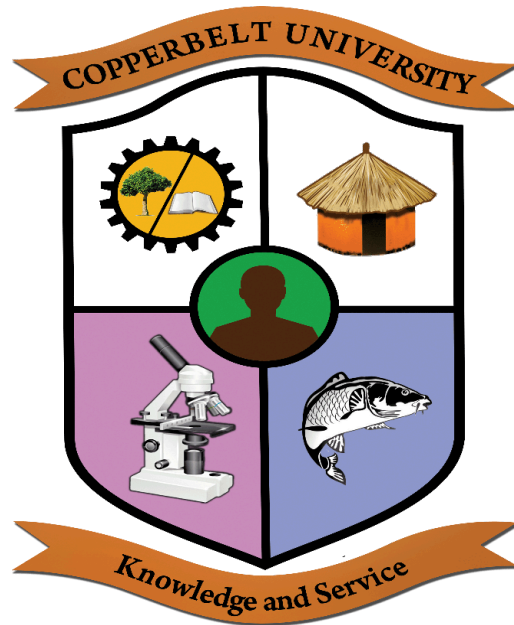




THE COPPERBELT UNIVERSITY

School of Information & Communication Technology

Case Study: Copperbelt University Library Management System

Programme: BSc in Information Systems

Group: The System Analysts (Group 2)

Course: System Analysis & Design

Course Code: 230

Table of Contents

Executive Summary.....	3
2. Object Oriented Concept Justification	4
2.1. Abstraction.....	4
2.2 Encapsulation.....	4
2.3 Inheritance.....	5
2.4 Polymorphism.....	5
2.5 Why Object-Oriented Design is Better Than Procedural Design.....	6
3. Use Case Diagrams.....	7
3.1 Actors and Use Cases.....	7
3.3 Detailed Use Case Descriptions.....	8
4. Class Diagrams.....	11
Task 5: Key Relationships Analysis.....	13
Task 6: Process Recommendation.....	14
Task 7: From Use Cases to Classes – The Noun-Verb Method.....	17
8. Task 8: Package Diagram.....	19
Task 9: Critical Reflection Memorandum.....	21
Team Contribution Statement.....	23

Executive Summary

Prepared for:

Dr. Musonda

University Librarian

Copperbelt University

The current library system at the Copperbelt University largely relies on manual processes which of these include physical borrowing records and manual fine calculations. These processes increase administrative workload, limit students' ability to access proper library services, and above all, create delays.

There has been an object-oriented system analysis conducted by our team based on the requirements gathered from the stakeholders; students, professors, librarians and administrators. The analysis focused mainly on understanding user needs, pointing out or identifying the system processes and modelling the relationships between major system components.

This analysis clearly shows the need for implementing an integrated digital system that is capable of supporting online book searches, reservations, loan management, automated fine calculations, borrowing history tracking and administrative reporting. The system must also support different user roles and multiple campus libraries.

We recommend the development of a centralized object-oriented library management system using an iterative development approach preferably, the Agile methodology. The system can support different users accessing library services online at the same time hence, improving operational efficiency, reducing paperwork, enhancing user experience providing accurate reporting and tracking capabilities for the university library, thereby, promoting data integrity.

2. Object Oriented Concept Justification

To build a modern Library Management software system that stands the test of time, we must look at software design differently. Traditional programming often reads like a rigid, step-by-step recipe. Object-Oriented (OO) design, on the other hand, models software after real-world things (objects). These objects could include a book, student, librarian, or loan record. With each object containing data and actions related to it. Below is an explanation of the four pillars of object-oriented design and how they protect the library's technical investment.

2.1. Abstraction

Abstraction means showing only the important features while hiding unnecessary details. In a library management system, a student only needs to see:

- Available books
- Borrowing history
- Reservations
- Fines

The student doesn't need to know how the database stores information internally. Take for instance a "search book" function; it allows users to search by title or author without understanding how data retrieval works behind the scenes.

Therefore, abstraction simplifies system usage and reduces complexity for users and developers.

2.2 Encapsulation

Encapsulation means keeping data and methods together while protecting data from unauthorized access. This is achieved by making a class attributes private and allowing access only through controlled methods.

This can be seen in a fine class, which may contain:

- Amount

- Due date

Instead of user randomly altering the fine amount, the system controls calculations using methods. This improves security and prevents accidental corruption or unauthorized access.

2.3 Inheritance

Inheritance in a library management system is used when different classes share common characteristics while maintaining their own features. It allows one class (subclass / child class) to inherit properties and methods from another class (parent class/ super class). By way of example, we can create general user (library user/ person) that holds common attributes and methods like:

- User ID
- Name
- Email
- Login
- Logout

Then we have:

- Student
- Professor
- Librarian

So, in this case the general user is the parent class / Super class, and the Student, Professor and Librarian are the Sub Classes. The Sub Classes inherit the attributes and methods from the general user. Thereby reducing repetition and improving maintainability.

2.4 Polymorphism

Polymorphism allows one action to behave differently depending on the object involved.

Suppose library users can calculate fines. We can consider creating a common action or method to calculate the fine. But different users may calculate fines differently. For example:

- Student pays full time after 10 days
- Professor pays after 25 days
- Librarians may not pay fines

Even though they all use the calculate fine action, the result would change depending on the object. Therefore, polymorphism increases flexibility and allows the system to handle different scenarios differently.

2.5 Why Object-Oriented Design is Better Than Procedural Design

Traditional procedural programming systems are built with long sequences of instructions that share the same data. This makes the system harder to manage. If one business rule changes, such as changing the late fine rate in a library system, the change can accidentally affect other parts of the program and cause errors in unrelated sections. In object-oriented programming, related data and behaviour are grouped into objects, making the system easier to maintain, update, and organize.

Aspects	Object-Oriented	Traditional Procedural
Encapsulation	Keeps data and methods together inside one object, improving security and preventing unauthorized access to data.	Data is often stored globally, so many functions can access and change it, increasing the risk of errors.
Inheritance	Allows classes to reuse existing code, reducing duplication.	Code is often repeated because there is no inheritance mechanism for sharing features between related components.
Polymorphism	Allows one method to behave differently for different objects, making systems flexible.	Separate functions must usually be written for every situation, making programs longer and harder to maintain.

Aspects	Object-Oriented	Traditional Procedural
Abstraction	Hides unnecessary details and shows only important features to the user.	Users and programmers may need to deal with many low-level details, making the system more complex.
Reusability	Classes and objects can easily be reused in other systems.	Reusing code is harder because procedures are tightly connected to specific program structures.
Scalability	New classes and features can be added without greatly affecting existing code.	As the program grows, the code becomes difficult to manage because many procedures depend on each other.
Maintainability	Changes can be isolated to one class without disturbing the whole system.	Small change in one function may affect unrelated sections of the program because data and functions are shared.
Real life modeling	Objects naturally represent real-world entities like Books, Members, and Librarians.	Programs focus more on processes than real-world entities, making large systems harder to design logically.

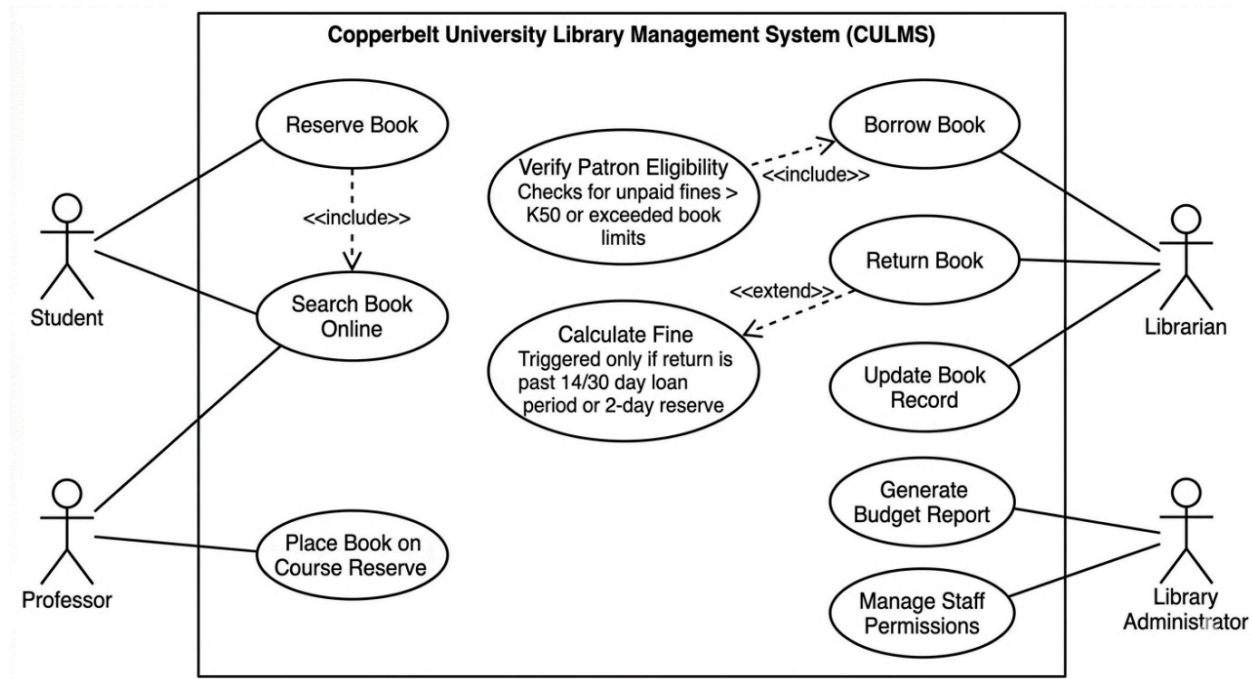
Therefore, for the Library Management System, Object-Oriented Design is better because the system contains many related entities such as Students, Books, professors, and librarians that interact continuously.

3. Use Case Diagrams

3.1: Actors and Use Cases

Actor Name	Description	Core Goals	Initiated Use Cases
Student	A university student enrolled at any campus who borrows standard and course reserve materials.	Find books online, check real-time availability, reserve checked-out books, track personal history/fines, and renew loans.	1. Search Book Online 2. Reserve Book 3. Renew Book Loan
Professor	A faculty member who requires extended borrowing privileges and manages academic reading lists.	Place books on 2-day course reserve, request new titles for collection, and track student reading engagement.	1. Place Book on Course Reserve 2. Request New Acquisition
Librarian	Library front-desk staff responsible for physical material handling and catalogue updates across campuses.	Automate checkouts/returns, issue late reminders, manage cross-campus library records, and modify catalogue data.	1. Borrow Book 2. Return Book 3. Update Book Record
Library Administrator	High-level library manager overseeing operations, personnel, and institutional integration.	Monitor budget metrics, view fine collections, adjust staff permissions, and track stats for institutional accreditation.	1. Generate Budget Report 2. Manage Staff Permissions

3.2: Use Case Diagram



3.3 Detailed Use Case Descriptions

3.3.1 Use Case 1: Borrow Book

1. **Use Case Name and ID:** Borrow Book

2. **Primary Actor:** Librarian

3. **Stakeholders and Interests:**

- Librarian: Wants to check out items accurately and quickly without manual physical ledger paperwork.
- Patron (Student/Professor): Wants an instant loan confirmation without manual errors.
- Dr. Musonda (University Librarian): Wants digital automation to minimize lost materials and log trends.

4. **Preconditions:** The Librarian has successfully authenticated into the CULMS terminal; the Patron presents a valid physical university ID card.

5. **Postconditions:** The system updates the status of the BookCopy to "On Loan"; the system links a new LoanRecord to the PatronAccount; the system reduces the Patron's remaining borrowing allowance.

3.3.1.1 Normal Flow (8 Steps):

- The Librarian scans the patron ID card barcode into the system card scanner.
- The System fetches the active patron record from the Student Information System database.
- The System displays the current loan count and fine balance on the monitor screen.
- The Librarian scans the unique asset barcode from the physical book copy.
- The System verifies the availability status of the designated book copy.
- The System creates a new loan ledger entry with the calculated calendar due date.
- The System alters the catalogue status of the book copy to "On Loan".
- The Librarian hands the physical book copy over to the waiting patron.

3.3.1.2 Alternate Flows

- AF 1 (Patron is a professor): At Step 6, if the system identifies the actor type as a Professor, the System alters the loan due date parameter to 30 days instead of 14 days.
- AF 2 (Book is a Course Reserve copy): At Step 6, if the system identifies the item classification as a Course Reserve, the System applies a strict 2-day non-renewable loan boundary.

Exception Flows

- EF 1 (Unpaid Fines Limit Exceeded): At Step 3, if the patron's unpaid fine balance surpasses K50, the System throws a "Borrowing Blocked" alert error, and the System terminates the current transaction.
- EF 2 (Borrowing Cap Exceeded): At Step 3, if a student currently holds 5 books (or a professor holds 15), the System displays a "Maximum Limit Reached" restriction error, and the System terminates the use case.

3.3.2 Use Case 2: Reserve Book

1. Use Case Name and ID: Reserve Book

2. Primary Actor: Student

3. Stakeholders and Interests:

- Student: Wants to guarantee access to high-demand, checked-out learning books online across campus networks.
- Librarian: Requires precise automated holds tracking to eliminate human friction or physical ledger errors.

4. Preconditions: The student is securely authenticated via the CULMS web portal; the chosen book title has zero available unborrowed physical copies.

5. Postconditions: The system saves a new pending ReservationRecord locked to the specific book copy; the book copy cannot be checked out by other users on return.

Normal Flow (7 Steps):

- The student inputs the desired book title query text into the search engine field.
- The System displays a filtered list of matching book entries on the web interface.
- The student triggers the online "Reserve" link element next to the selected checked-out title.
- The System reviews the student's active hold counts and financial outstanding liabilities.
- The System logs a new reservation entry containing a strict 7-day expiration timestamp.
- The System modifies the specific book copy queue index to "Reserved".
- The System outputs a visual reservation confirmation code onto the client browser.

3.3.2.2 Alternate Flows:

- AF 1 (Book Copy is Returned Early): If a running loan is cleared by a librarian before the reservation timeline expires, the System automatically dispatches an SMS email notice to the holding Student.
- AF 2 (Multiple Copies Exist): At Step 3, if several copies of the book exist on loan, the System assigns the reservation flag directly to the copy showing the closest absolute calendar return date.

3.3.2.3 Exception Flows:

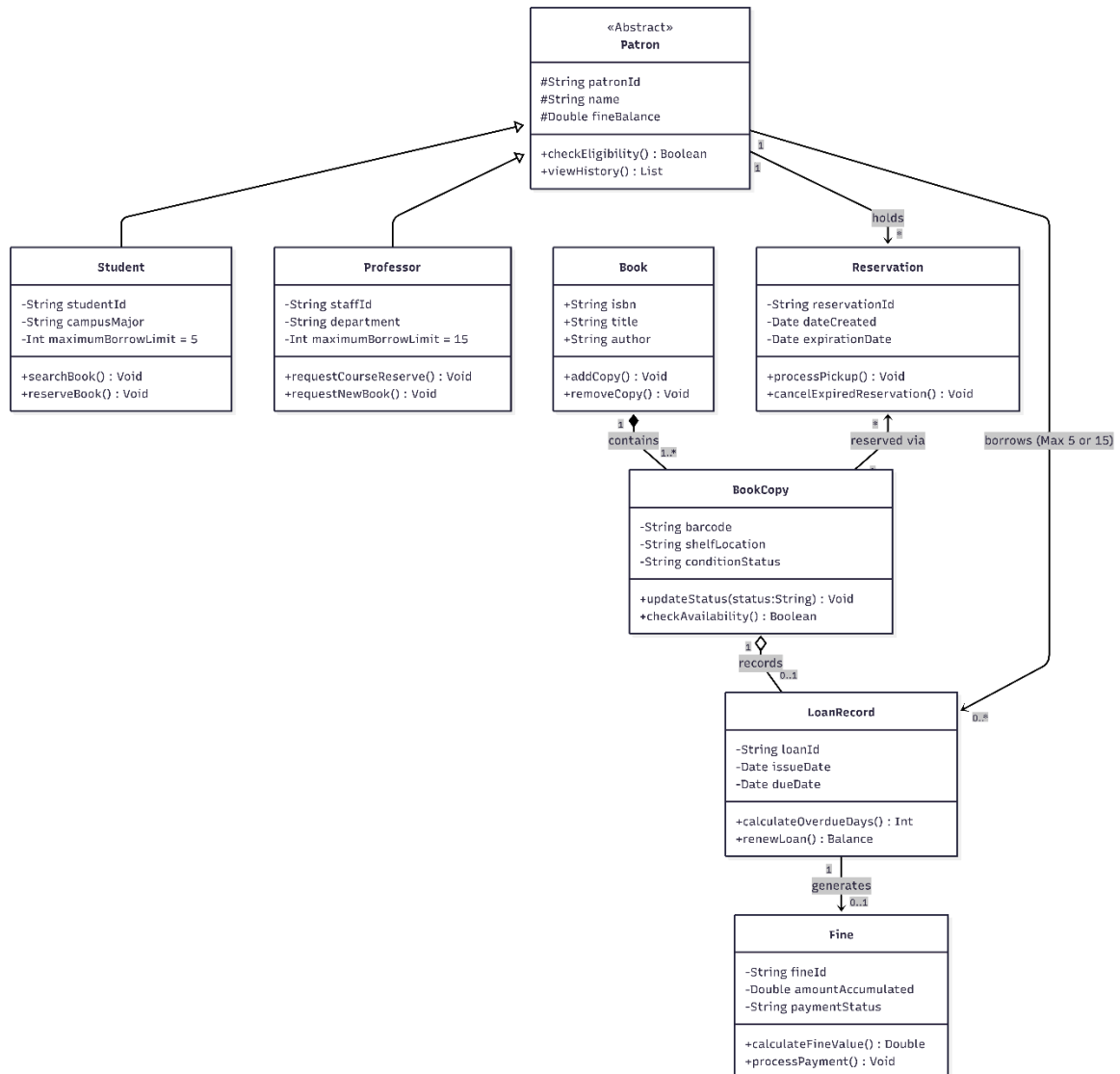
- EF 1 (Item Has Available Physical Copies): At Step 4, if the system analysis engine detects an available physical unborrowed copy on shelves, the System displays an error

message telling the student to pick up the book in person, and the System cancels the reserve request.

- EF 2 (Account Penalties Hold): At Step 4, if the student's active fine ledger holds a debt exceeding K50, the System presents a "Reservation Prohibited" warning block, and the System stops execution.

4. Class Diagrams

4.1 UML Class Diagram



4.2 UML Class Diagram

1. The Inheritance Hierarchy and Design Justification

The structural model utilizes a Generalization (Inheritance) hierarchy to represent system users. The Patron class serves as an abstract base class, containing shared attributes and methods common to all borrowers, such as `patronId`, `name`, `fineBalance`, and the `checkEligibility()` method.

The Student and Professor classes inherit directly from Patron. This design was chosen to enforce the distinct business rules assigned to different user types while eliminating redundant code. According to the system requirements, a student is restricted to a maximum of 5 books with a standard 14-day loan period, whereas a professor can borrow up to 15 books with a 30-day loan period. By using inheritance, the system can dynamically apply polymorphism—allowing the shared `checkEligibility()` method to process these differing constraints automatically based on which specific subclass is initiating the transaction.

2. Composition Relationship: Book and BookCopy

The diagram features a strict Composition relationship between the Book class and the BookCopy class. This is represented by a filled diamond on the Book side.

This relationship is composition—rather than standard aggregation—because of strict lifecycle dependency. The Book class represents the conceptual catalogue entry (containing the ISBN, title, and author). The BookCopy class represents the actual physical items sitting on the shelf, identified by unique barcodes. If the library permanently removes a Book title from its catalogue database, all corresponding BookCopy records must logically be destroyed alongside it; a physical barcode record cannot exist in the system without a parent catalog entry defining what the book is.

3. Association Relationship and Multiplicity

A Primary Association relationship exists between the Patron class and the LoanRecord class to track active borrowing. The multiplicity on this association is defined as 1 on the Patron side and 0..5 (for Students) on the LoanRecord side. This indicates that exactly one unique patron is financially and administratively responsible for a specific loan. The 0..5 constraint directly models the known business rule that a student may have zero currently borrowed books (0), but cannot exceed a maximum of 5 active books at any given time. For the Professor subclass interacting with this association, the multiplicity shifts to 0..15 to accommodate their expanded borrowing limit.

Task 5: Key Relationships Analysis

Campus University Library Management System, here are 3 key relationships:

1. LibraryMember and Loan – Association

This is an association relationship. We chose association over composition because a LibraryMember can exist independently of any Loan, and a Loan can be ended without deleting the member. If we used composition, deleting a member would incorrectly delete all their loan history, which is needed for records. The business rule is that a member can borrow multiple books over time, and each loan record links one member to one book copy. The multiplicity is 1 LibraryMember to 0..Loans. It is correct because a member may have zero loans now, but can have many over time, while each loan belongs to only one member.

2. Book and BookCopy – Composition

This is a composition relationship. We chose composition because a BookCopy cannot exist without its parent Book, and it represents a physical instance of that book title. If the Book record is deleted from the system, all its copies should also be removed. The business rule is that a library holds multiple physical copies of a book title, but those copies are meaningless without the title metadata. The multiplicity is 1 Book to 1..BookCopies. It is correct because a title must have at least one copy to exist in inventory, and each copy belongs to exactly one title. This enforces strong ownership and lifecycle dependency.

3. Person and Librarian/Student – Generalization

This is a generalization relationship. We chose generalization because both Librarian and Student share common attributes like name, ID, contact information of a Person, but have different roles and behaviors. Using generalization avoids duplicating attributes and follows OOP inheritance principles. The business rule is that all library users are people, but only some are staff with management privileges. The multiplicity is not directly used here, but a Person can be 0..1 Librarian and 0..1 Student. It is correct because it models real-world "is-a" relationships and makes it easy to add new user types later.

Task 6: Process Recommendation

Recommended Development Process

Our team recommends using Agile (Scrum framework) as the software development process for the CULMS project. Given the 4-month delivery window and the fact that Dr. Musonda herself admitted the requirements are still evolving, we felt Waterfall or a fully plan-driven approach would put the project at risk before it even started.

Scrum divides work into short sprints of two weeks each, which means the team is always delivering something working and getting feedback along the way rather than waiting until the very end to find out if the product meets expectations.

Justification (4 Reasons Tied to Project Characteristics)

Reason 1 – Requirements Are Incomplete and Will Change

The assignment brief states clearly that librarians will likely request new features once they see a working version of the system. This is almost the definition of a project that should not be managed under Waterfall, where requirements are locked in before development begins.

Scrum handles evolving requirements through its product backlog. New feature requests from librarians can be added to the backlog after each sprint review, assessed for priority by the product owner, and slotted into the next available sprint without disrupting work already done.

Reason 2 – The Use Cases From Task 3 Map Naturally to Sprint Deliverables

When we look at the use cases we identified in Task 3, it is clear that each one represents a self-contained, shippable feature. For example, Borrow Book and Reserve Book are complete end-to-end transactions with their own preconditions, flows, and exception handling. These can each be developed and tested within a single sprint.

The actors we identified (Student, Librarian, Professor, Library Administrator) also represent different user groups who can be brought in at sprint reviews to give feedback. This is much more effective than waiting for a full system demonstration four months from now.

Reason 3 – The 4-Month Timeline Suits Sprint-Based Delivery

Four months gives the team roughly 8 two-week sprints. This is enough time to cover the core borrowing and reservation flows in early sprints, then build out fines management, reporting, and admin features in later sprints, and still leave a final sprint for user acceptance testing and deployment. A Waterfall project with the same timeline would spend weeks on documentation before writing a single line of code.

Reason 4 – Team Size Is Ideal for Scrum

The team has 5 analysts and developers, which falls exactly within the recommended size for a Scrum team of 3 to 9 members. Smaller teams communicate with less friction, and daily stand-up meetings stay short and focused. A larger team or a different process like RUP would introduce coordination overhead that does not match the scale of this project.

Why Scrum Specifically

Within the Agile family we chose Scrum over Kanban and XP for the following reasons:

- Scrum has defined roles (Product Owner, Scrum Master, Development Team) which gives the team and Dr. Musonda a clear structure for communication and decision-making.
- Kanban is more suited to continuous maintenance work rather than building a new system from scratch with a fixed delivery deadline.
- XP is focused heavily on pair programming and test-driven development which can slow down a small team working under a 4-month constraint unless the team is already experienced with it.
- Scrum sprint reviews give stakeholders like the librarians and library administrator a regular checkpoint every two weeks to see progress and redirect the team early if something is off.

One Risk of Using Scrum and How to Mitigate It

Risk: Scope Creep From Evolving Stakeholder Requests

Because the requirements are still unclear and librarians have already signalled they will request new features once they see the system, there is a real risk that the backlog keeps growing throughout the project. If every new idea gets added and prioritised the team could end up in a cycle of never completing a solid release.

Mitigation

- Assign Dr. Musonda or a senior librarian as the Product Owner with the authority and responsibility to prioritise and freeze the sprint backlog before each sprint starts.
- Use a definition of done for each sprint so that partially completed features cannot carry over indefinitely.
- At the start of the project define a minimum viable product (MVP) covering the core use cases from Task 3 (Borrow Book, Return Book, Reserve Book, Calculate Fine) and protect those features from being deprioritised in favour of new requests.
- Any new feature requests go into an ice box section of the backlog and are only promoted after the MVP sprint goals are confirmed as achieved.

Project Timeline – 4-Month Gantt Overview

The table below shows how the 4-month period is divided across sprints. Each sprint is two weeks long, giving 8 sprints total. The final two weeks are reserved for user acceptance testing and go-live.

Phase / Sprint	Wk 1-2	Wk 3-4	Wk 5-6	Wk 7-8	Wk 9-10	Wk 11-12	Wk 13-16
Sprint 1 Project Setup & Backlog Key work: environment setup, finalise backlog, define MVP, assign roles	•						

Phase / Sprint	Wk 1-2	Wk 3-4	Wk 5-6	Wk 7-8	Wk 9-10	Wk 11-12	Wk 13-16
Sprint 2 Core Borrowing ModuleKey work: Borrow Book, Return Book use cases (Task 3 UC1)		•					
Sprint 3 Reservation & SearchKey work: Reserve Book use case (Task 3 UC2), book search			•				
Sprint 4 Fines & RenewalKey work: Calculate Fine, Renew Loan, automatic fine reminders				•			
Sprint 5 Course Reserve & CatalogueKey work: Place Book on Course Reserve, Update Book Record, multi-campus support					•		
Sprint 6 Admin, Reports & IntegrationKey work: Budget reports, staff permissions, student info system integration						•	
UAT & Go-Live (Wk 13-16)Key work: User acceptance testing with librarians, bug fixes, staff training, deployment							•

Task 7: From Use Cases to Classes – The Noun-Verb Method

Part A: Noun Extraction

From the Borrow Book use case, the following nouns were identified and categorized:

Noun	Category	Notes
Student	Class	Represents the library user borrowing books
Librarian	Class	Staff member who facilitates borrowing
Book	Class	General representation of a book
BookCopy	Class	Specific copy of a book with barcode
Barcode	Attribute	Belongs to BookCopy
Loan	Class	Represents the borrowing transaction
Fine	Class	Linked to overdue loans
System	Class	The LMS itself handling processes
Borrowing History	Attribute	Belongs to Student
Reservation	Class	Represents reserved books
Due Date	Attribute	Belongs to Loan
Availability	Attribute	Belongs to BookCopy
Notification	Class	Used to alert students about due dates/fines
Campus Library	Class	Represents different library branches
ID Card	Attribute	Belongs to Student

Part B: Verb-to-Method Mapping

From the same use case, the verbs (actions) were mapped to classes and methods.

Verb/Action	Performed by Class	Method Name	Receiver Class
scans barcode	Librarian	scanBarcode()	BookCopy
checks availability	System	checkAvailability()	BookCopy
verifies student ID	System	verifyStudentID()	Student
records loan	System	recordLoan()	Loan
updates borrowing history	System	updateHistory()	Student
sets due date	System	setDueDate()	Loan
prints receipt	System	printReceipt()	Student
calculates fine	System	calculateFine()	Loan
sends notification	System	sendNotification()	Student
updates book status	System	updateStatus()	BookCopy

Explanation

To derive these tables, I analysed the normal flow of the Borrow Book use case.

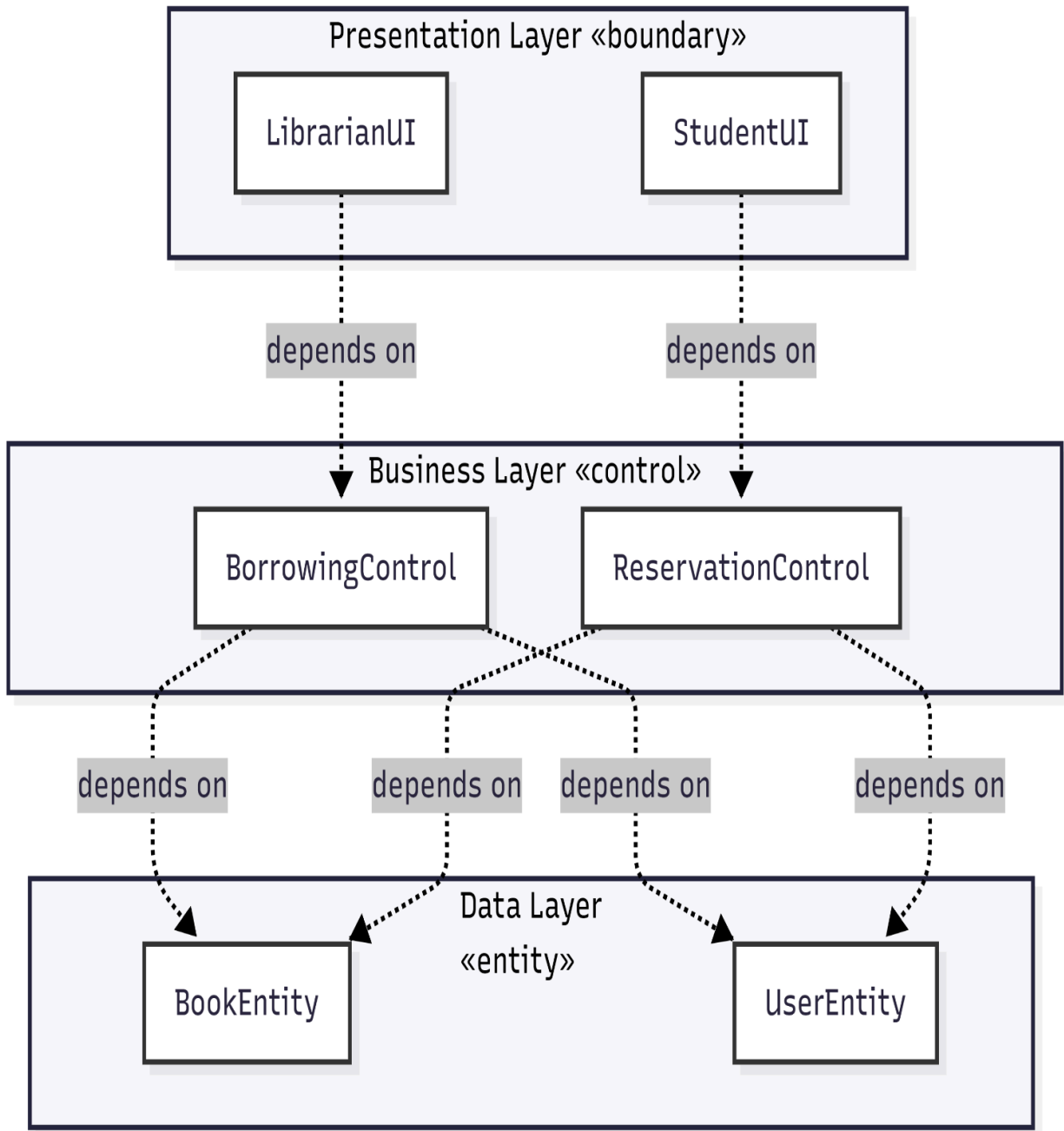
In Part A, I extracted all the relevant nouns and determined whether they should function as a Class or an Attribute within the system architecture. For instance, "Barcode" does not need to be an independent class, so it was categorized as an attribute belonging to BookCopy.

In Part B, I isolated the key verbs and actions to map them to functional methods. Each action was assigned a performing class and a receiving class to establish clear relationships. For example, when the librarian scans a barcode, the Librarian class invokes the scanBarcode()

method on the BookCopy class. This systematic approach ensures that textual requirements transition smoothly into structured elements for the UML model.

8. Task 8: Package Diagram

8.1 Copperbelt University Library Management System Package Diagram



8.2 Library Management System Package Diagram Deliverable

Why the Layers Are Organized This Way:

The architecture for CULMS is organized into three classic software layers: Presentation, Business, and Data, following software engineering best practices for separation of concerns. This approach keeps the system's core responsibilities distinct and manageable. The Presentation Layer (using boundary objects like StudentUI) is solely responsible for user interaction and interface, handling how information is displayed to different user roles. The Business Layer (using control objects like BorrowingControl) encapsulates all the system's rules, logic, and transaction processing. Finally, the Data Layer (using entity objects like BookEntity) handles all data storage, persistence, and representation of core database entities. This creates a clean boundary between "what the user sees," "how the system thinks," and "what information the system stores."

The Dependency Rules:

A core rule of layered architecture is strict one-way dependency. Upper layers depend on lower layers to fulfill their tasks, but lower layers have absolutely no knowledge of the layers above them. In the provided diagram, for example, the LibrarianUI package in the Presentation layer depends on the BorrowingControl package in the Business layer to perform book borrowings. Conversely, the BorrowingControl package does not depend on, or even know about, any UI package; it merely executes logic and uses Data Layer packages. This unidirectional dependency prevents circular dependencies, which makes the system easier to compile and understand. For instance, the prompt's rule holds true: "Presentation depends on Business, not the other way."

How This Architecture Supports Maintenance and Scalability:

- **Maintenance:** Maintenance is greatly simplified. Changes can be isolated to a single layer without propagating throughout the entire system. For example, if the library decides to change its underlying database from an SQL system to a NoSQL one, modifications would be confined to the Data Layer, leaving the Business logic and User Interfaces largely untouched. This minimizes the risk of introducing new bugs in unrelated parts of the application. Furthermore, individual layers and packages can be tested in isolation.
- **Scalability:** Each layer can be scaled independently based on its specific demands. If there is a huge increase in users performing searches, the application server running the

Presentation Layer can be scaled up. If complex reports need to be generated frequently, the servers running the Business logic can be fortified. This modular approach is much more efficient than having to scale the entire monolithic application to meet a localized demand, thus saving resources and improving performance.

Task 9: Critical Reflection Memorandum

MEMORANDUM

To: Dr. Musonda, University Librarian

From: Systems Analysis Team

Date: 29th May, 2026.

Subject: Critical Reflection – Limitations of CULMS Analysis

This memo presents a critical reflection on the analysis conducted for the proposed Copperbelt University Library Management System (CULMS). While our team has developed a strong initial analysis model, we want to be transparent about the limitations of our current analysis before moving to detailed design and development.

1. Two Assumptions That May Not Be Fully Accurate

Assumption 1: Network and Hardware Infrastructure Is Ready

Our design assumes that each campus has reliable internet, sufficient bandwidth, and computers for staff and public use. This was not confirmed in the requirements gathering. If some campuses have slow connections or outdated hardware, features like real-time online reservations and multi-campus reporting may be slow or unusable. This would affect user adoption and staff efficiency. We need to validate the IT environment before finalizing technical requirements.

Assumption 2: User Roles and Loan Policies Are Standard Across Campuses

We assumed that loan limits, fine rules, and user roles are the same for the main campus, medical school, and engineering library. In practice, faculty in the medical school may need longer loan periods, and fines may differ by department. If policies vary, our current class model and business rules will need adjustment. Building the system on a "one-size-fits-all" assumption risks rework after launch.

2. One Open Question We Need Answered Before Design

Question: What is the required integration level with the University Student Information System [SIS]?

The case study mentions that the system should "integrate with the University's student information system for user authentication and data synchronization." We do not know if this means:

- Real-time, automatic sync of student and staff records,
- Daily batch import/export, or
- Manual data entry with occasional checks.

This decision impacts system security, data accuracy, and development effort. Real-time integration is more complex and may require IT department involvement. Please clarify the expected level of integration so we can design the correct interfaces and avoid delays later.

3. Missing Potential Requirement

Need for Backup and Recovery

One potential requirement that we may have missed during the initial stakeholder interviews is the need for an automated backup and recovery functionality. In case of system failure, there should be a quick way to retrieve all data that might get lost. Since the proposed system will store important information such as borrowing records, fines, reservations, and personal user details, the loss of data due to hardware failure, cyberattacks, or power outages could seriously disrupt library operations. Including backup and retrieval mechanisms would help ensure data security, business continuity, and system reliability.

4. Risk that could lead to wrong analysis

Changing Requirements

A major risk that could affect accuracy of the current analysis model is the possibility of changing stakeholder requirements during later stages of development. As librarians, students and administrators interact with the early system, they may request additional features and modifications that were not identified during initial analysis phase. These changes could affect

the existing use cases, class relationships, and business rules, resulting in parts of the analysis becoming unreliable or incomplete. Hence continuous stakeholder feedback as well as iterative review sessions will be necessary to ensure that the model remains accurate and aligns with user needs.

Although the current analysis provides a strong foundation for the proposed Copperbelt University Library Management System (CULMS), further stakeholder consultation, continuous feedback and validation will be necessary as the project progresses into the actual design and development phases. Addressing the identified limitations and risks early will help ensure that the final system is reliable, scalable, and capable of effectively supporting the academic and administrative needs of the Copperbelt University Library.

Team Contribution Statement

To promote fairness and to ensure balanced workloads during every phase of the project, our group assigned tasks strategically based on individual analytical skills.

Group Member	Student ID	Assigned Task
RUTH MULENGA	24155619	Task 1 & 9 (Executive summary & Critical reflection)
LUKWESA KHONDOWE	24144475	Task 2 & 10 & 8 (Object-oriented concept justification & Team contribution statement)
SANDE MUKOSHA	24165705	Task 3 & 4 & 8 (Use case model & Class diagram)
NAMAKAU MOTO	24999202	Task 5 & 9 (Relational Analysis & Critical reflection)
CHISHIBE KABWE	24148580	Task 6 (Task Process Recommendation)
MANDLA PEME	23136556	Task 7 (The Noun-Verb Method)

Collaboration

We held weekly or rather daily online meetings using WhatsApp. Furthermore, we used Microsoft word to write our text requirements and visual paradigm to draft our UML class diagrams.

Challenges

Because the assignment was distributed close to the recess period, most group members had already traveled to their respective homes. As a result, the group was unable to meet or collaborate physically and therefore relied mainly on online meetings for communication and coordination.

Additionally, when we partly disagreed on whether to use the iterative adaptive development process or agile development approach, we setup a short review session to look over our lecture notes and resolve the issue together based on the project business rules.

Confirmation of Contribution

By signing below, all group members verify that the work division was balanced and fair, and that everyone contributed approximately equally to the project.

Member 1: Ruth Mulenga

Member 2: Lukwesa khondowe

Member 3: Sande mukosha

Member 4: Namakau moto

Member 5: Chishibe Kabwe

Member 6: Mandla peme